My assigned project is a Fruit Ninja-style game built using Python and Pygame. My original assigned repository was:
https://github.com/SETAPESU26/41_fruit-ninja
My personal repository is:
https://github.com/SruthiVejju/LAB4_FruitNinja_SruthiVejju_PES1UG24CS472-.git
explain how a Pygame game works, what the game loop does, and how files such as main.py, fruit.py, and game_engine.py work together

Worked for 14s

Understanding your Fruit Ninja game in Python + Pygame

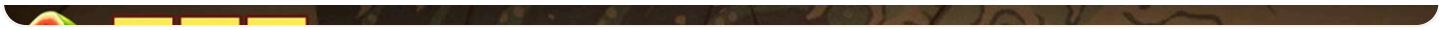
Your assigned project is a **Fruit Ninja-style game built using Python and Pygame**. The easiest way to understand it is to think of the game as three parts:

- `main.py` — starts the game and coordinates everything.
- `game_engine.py` — controls the game rules, movement, scoring, collisions, and overall gameplay.
- `fruit.py` — defines how individual fruits and bombs behave.

These files work together to create one interactive game. Pygame provides the window, graphics, mouse input, event handling, and timing; your Python code defines the actual game mechanics.

pygame v2.6.... +1

1. How does a Pygame game work?



Pygame is a Python library used to create 2D games. It provides functions to:

- Open a game window.
- Draw images, shapes, text, and backgrounds.
- Detect mouse and keyboard input.
- Play sounds.
- Update objects on the screen.
- Control how quickly the game runs.

In your Fruit Ninja game, imagine this sequence:

1. A fruit appears near the bottom of the screen.
2. The game gives it an upward velocity, making it fly.
3. Gravity gradually pulls it back down.
4. You move your mouse across the fruit to slice it.
5. The game detects the slice and updates your score.
6. If you miss a fruit, you lose a life.
7. If you hit a bomb, the game applies the bomb penalty.
8. When the game runs out of lives, it reaches the game-over state.

The game must continuously repeat these operations while you play. That repetition is called the **game loop**.

2. What is a game loop?

A game loop is a repeating block of code that keeps the game running until the player exits or the game ends.

Your project uses a loop that runs at approximately **60 frames per second (FPS)**. The main steps are handling events, processing input, updating the game, rendering the screen, and controlling the frame rate. pygame v2.6.... +1

1. Handle events
Check for window-close events and player input.

↓

2. Handle input
Read mouse movement and slicing actions.

↓

3. Update the game

Move fruits, apply gravity, check collisions, update score and lives.

↓

4. Render the screen

Draw fruits, bombs, the blade trail, score and lives.

↓

5. Control FPS

Use the clock to limit execution speed, then repeat.

The loop continues until the game exits.

Why is the game loop necessary?

Suppose the game draws a fruit only once. That fruit would not move, react to your mouse, or fall under gravity.

The loop repeatedly updates the fruit's position and redraws it. This creates the illusion of continuous movement.

A simplified version looks like this:

</> Python

```python
running = True

while running:
    # 1. Read events
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # 2. Process player input
    game.handle_input()

    # 3. Update game logic
    game.update()

    # 4. Draw everything
    game.render()

    # 5. Limit frame rate
    clock.tick(60)
```

This is a simplified illustration of the structure, not a replacement for your project's actual code.

Important distinction: `update()` changes the game's state; `render()` displays that state. For example, `update()` moves a fruit from one position to another, while `render()` draws it at its new position.

3. How your three Python files work together

Your project structure is organized like this:

- **LAB4_FruitNinja/**
  - `main.py`
    Entry point — starts and coordinates the game.
  - `game/`
    - `game_engine.py`
      Game rules, spawning, physics, collisions, scoring and rendering.
    - `fruit.py`
      Individual fruit/bomb objects and their movement and collision behavior.
  - `requirements.txt`
    Lists the Python packages needed to run the game.

A. `main.py` — the starting point

When you execute:

```bash
python main.py
```

Python begins running `main.py`.

Its job is to initialize Pygame, create the game window, create the game engine, and start the game loop.

The main responsibilities are:

1. Initialize Pygame using `pygame.init()`.
2. Set the window dimensions, such as 700 × 600 pixels in your project.
3. Create the `GameEngine` object.
4. Listen for events, including closing the window.
5. Call the engine's input-handling, update, and rendering methods.

6. Control the frame rate using a Pygame clock.
7. Shut down Pygame when the game exits.

The important idea is that `main.py` does not need to contain all the fruit physics and collision logic itself. It delegates those tasks to the engine.

B. `game_engine.py` — the brain of the game

The game engine coordinates what happens during gameplay.

In your project, its responsibilities include:

- **Spawning:** creating fruits and bombs at appropriate intervals.
- **Physics:** updating fruit positions and applying gravity.
- **Input:** reading the mouse's slicing movement.
- **Collision detection:** determining whether the mouse swipe intersects a fruit or bomb.
- **Scoring:** awarding points for sliced fruits.
- **Lives and game-over:** applying penalties and checking whether the game should end.
- **Rendering:** drawing game objects, the score, lives and the blade trail.

For example, when you slice a fruit, the engine determines whether the swipe intersects it and then updates the appropriate game state.

The engine also coordinates the objects defined in `fruit.py`. This separation keeps the code more organized than putting every feature into one enormous Python file.

C. `fruit.py` — individual game objects

This file defines the behavior of a fruit object and, depending on how your implementation is structured, the collision helpers it uses.

A fruit object generally needs information such as:

- Its position: x and y.
- Its velocity: horizontal and vertical movement.
- Its size or collision boundary.
- Its type: fruit or bomb.
- Its update and drawing behavior.

For example, a fruit might start with an upward velocity. Gravity then changes its vertical velocity on every update, causing it to rise and eventually fall.

A simplified physics example is:

</> Python

```
velocity_y += gravity
y += velocity_y
```

Here:

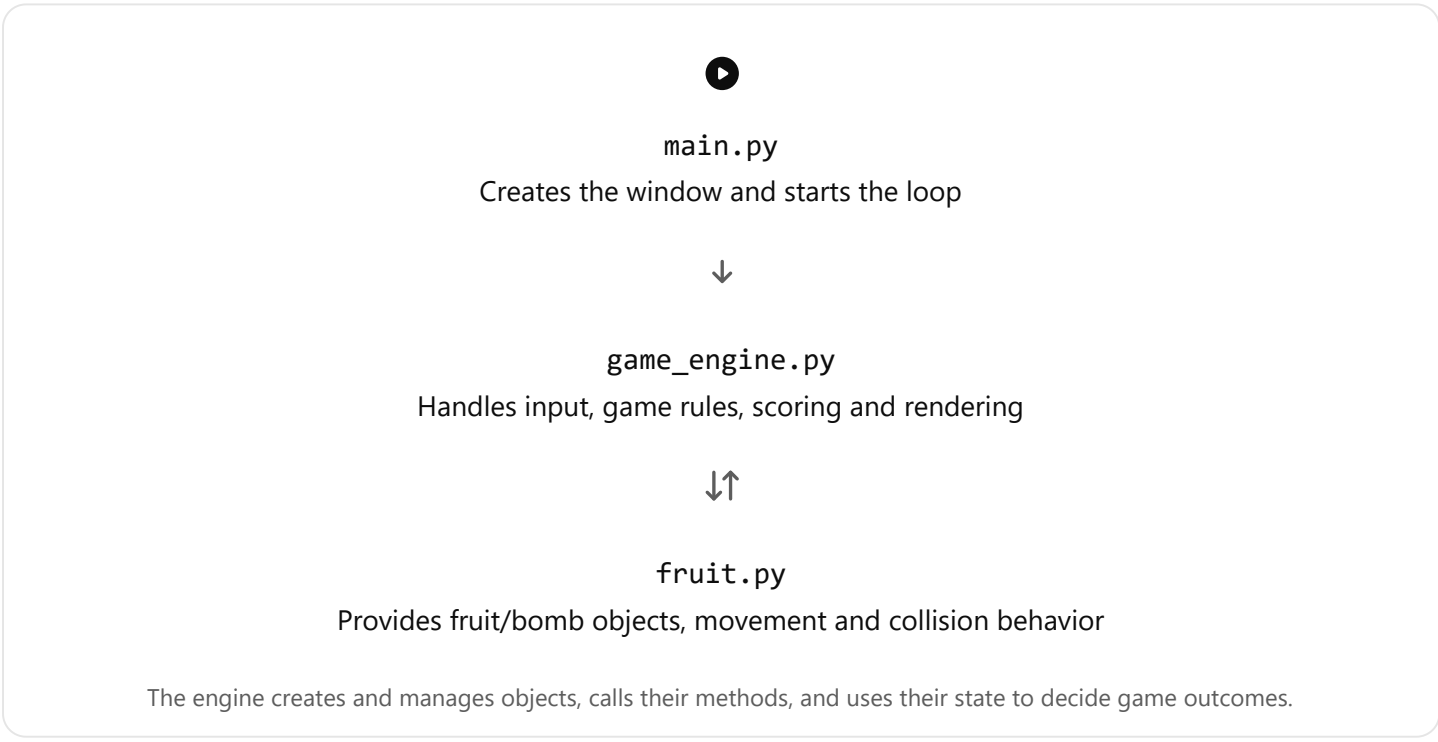
- `velocity_y` is the vertical velocity.
- `gravity` accelerates the fruit downward.
- `y` is the fruit's vertical position.

In Pygame, screen coordinates increase downward, so a negative vertical velocity usually means upward movement.

Your project also uses a segment-based collision check, `intersects_segment()`, to help detect a mouse swipe passing across a fruit between two successive mouse positions.

Why keep this in `fruit.py`? The fruit object can define how it moves and whether a swipe intersects it, while `game_engine.py` decides what the game should do when that collision happens.

4. How the files communicate



Consider what happens when you slice an apple:

1. `main.py` calls the engine's input-handling method.
2. `game_engine.py` reads the mouse positions and checks for a swipe.
3. The engine checks whether the swipe intersects the apple using the fruit's collision logic.
4. The engine marks the apple as sliced or removes it from the active objects.
5. The score increases.

6. The next rendering pass displays the updated score and no longer draws the intact apple.

If the swipe intersects a bomb instead, the engine applies the bomb's penalty rather than awarding fruit points.

5. Important concepts you should understand

Concept	Meaning in your game
Class	A blueprint for creating objects, such as a fruit.
Object	One particular fruit or bomb created from a class.
Instance variables	Data belonging to an object, such as its position and velocity.
Method	A function belonging to an object or class.
Event handling	Responding to mouse actions and window events.
Physics	Simulating movement and gravity.
Collision detection	Checking whether a swipe intersects a game object.
Rendering	Drawing the current game state onto the window.
FPS	Frames per second; your game targets 60 FPS.
Game state	Information such as score, lives and whether the game is over.

6. What happens when you slice a moving fruit?

There is a particularly important detail in your project: **the mouse can move a significant distance between frames**.

Imagine that a fruit is at position `(200, 300)`.

- At one moment, your mouse is at `(150, 300)`.
- At the next moment, your mouse is at `(250, 300)`.

The mouse has crossed the fruit, even if neither recorded mouse position is directly inside it.

A collision system that checks only the current mouse position might miss this slice.

Your project's `intersects_segment()` approach checks the line segment connecting the previous and current mouse positions against the fruit's collision area. This makes fast swipes more reliable.

This is an example of **continuous collision checking between sampled positions**, rather than checking only a single point. It is a useful implementation detail to understand if you are asked why the slicing logic is designed that way.

7. Quick viva questions

1. What is Pygame?

2. What is the game loop?

3. Why do we use main.py?

4. What is the role of game_engine.py?

5. Why separate fruit.py from game_engine.py?

6. What does gravity do?

7. What is collision detection?

8. Why use clock.tick(60)?

9. What does pygame.display.flip() do?

10. Why check the mouse segment instead of only its current position?

Reveal all answers

8. What you should remember most

If you remember only these three things, you will have the basic architecture clear:

- `main.py` **starts and coordinates the game.**
- `game_engine.py` **controls the rules and the overall gameplay.**
- `fruit.py` **defines the behavior of individual fruits and bombs.**

The game loop connects everything: it continuously reads input, updates objects and game state, draws the result, and repeats.

For further reading, the official Pygame documentation ↗ explains the event loop, display updates and frame-rate control. pygame v2.6.... +2

One important qualification: these explanations follow the project structure and behavior described above. I have not verified every current line of code in both of your GitHub repositories in this response, so use your actual implementation when answering questions about specific methods or exact logic.

> Explain the original collision detection problem in my Fruit Ninja game. Explain why fast mouse swipes can miss fruits, how line-segment collision detection works, and how the Fruit class and GameEngine work together to detect a slice. Include a simple example that I can explain during my lab evaluation.

Fruit Ninja: Collision Detection Explained

The main problem in your Fruit Ninja game is that **a fast mouse swipe can pass through a fruit without the game detecting the slice**. To understand why, you need to know how mouse input is checked during the game loop and how line-segment collision detection solves the problem.

1. What is the original collision detection problem?

Imagine that a fruit is flying across the screen and you quickly swipe your mouse through it.

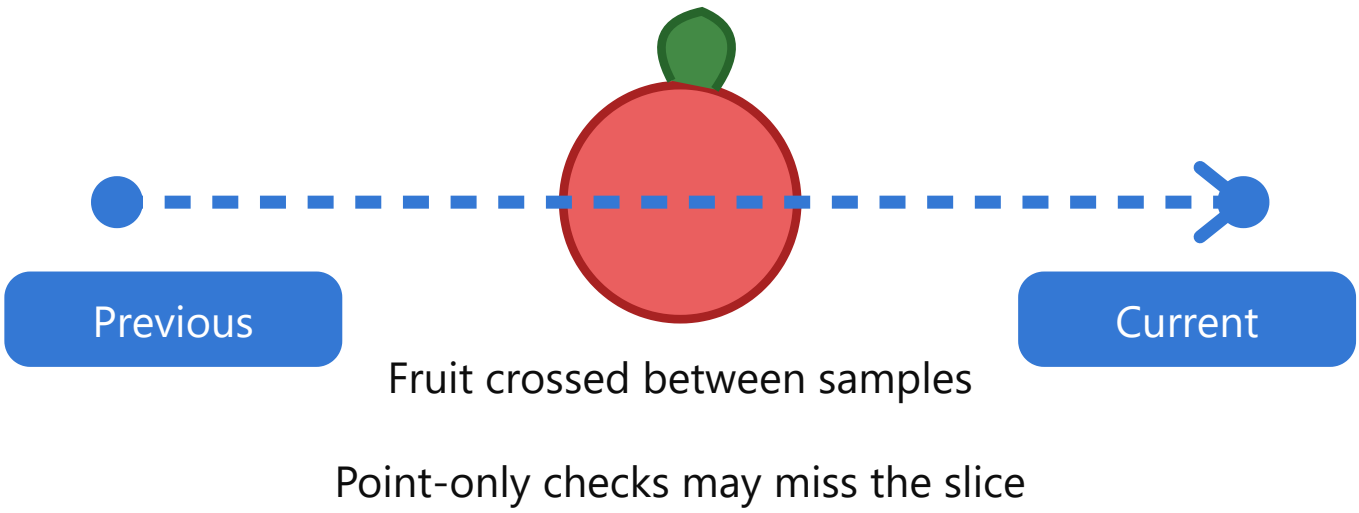
A simple collision detection approach might check whether the mouse's current position is inside the fruit's boundary.

For example:

- Previous mouse position: `(100, 200)`
- Current mouse position: `(300, 200)`
- Fruit center: `(200, 200)`

The mouse has moved straight through the fruit. However, neither the previous nor the current mouse position is inside the fruit.

Why the slice can be missed



The dashed line represents the mouse's movement. The fruit lies along that path, even though both sampled endpoints are outside it.

This happens because mouse positions are sampled at discrete intervals, typically once per game-loop iteration. If the mouse moves a large distance between two samples, checking only those two points can miss an object between them.

This is called a **missed collision**, and the solution is to check the path travelled by the mouse rather than just its final position.

2. How line-segment collision detection solves it

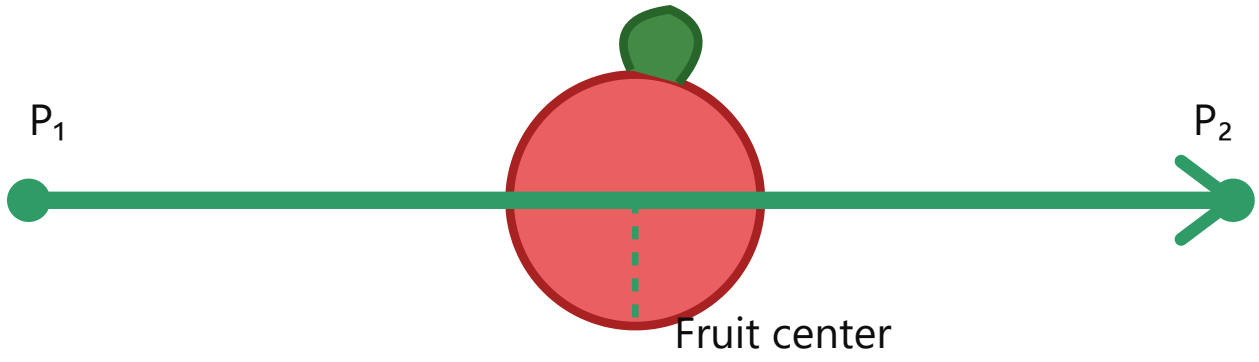
Instead of treating the mouse as a single point, we treat its movement between two consecutive positions as a **line segment**.

Let:

- $P_1 = (x_1, y_1)$ be the previous mouse position.
- $P_2 = (x_2, y_2)$ be the current mouse position.
- $C = (c_x, c_y)$ be the fruit's center.
- r be the fruit's collision radius.

The line segment from P_1 to P_2 represents the swipe made between frames.

The improved approach



Segment intersects fruit

Slice detected!

The game tests whether the mouse's path intersects the fruit's collision area. If it does, the fruit can be marked as sliced.

How does the mathematical check work?

For a circular fruit, we can find the closest point on the mouse segment to the fruit's center.

First, calculate the direction of the segment:

$$D = P_2 - P_1$$

Next, find how far along the segment the fruit's center projects:

$$t = \frac{(C - P_1) \cdot D}{D \cdot D}$$

Clamp t to the range from 0 to 1. This ensures that the closest point lies on the finite segment, not on an infinitely extended line.

Then calculate that closest point:

$$Q = P_1 + tD$$

Finally, calculate the distance from the fruit's center to Q:

$$d = \sqrt{(c_x - Q_x)^2 + (c_y - Q_y)^2}$$

The collision condition is:

$$\boxed{d \leq r}$$

If the distance is smaller than or equal to the fruit's radius, the segment intersects the circular collision area, so the game detects a slice.

In a real game, the fruit's collision radius may be smaller or larger than its visual size. Also, a robust implementation must handle a zero-length segment when the mouse has not moved.

3. How `fruit.py` and `game_engine.py` work together

The key distinction is that the **Fruit class helps determine whether a collision occurred, while the GameEngine manages what happens because of that collision**.

Step 1 — `fruit.py`

The Fruit class stores the fruit's information, such as position, velocity and collision boundary. Its `intersects_segment()` method can check whether the mouse's movement segment intersects the fruit.

↓

Step 2 — `game_engine.py`

The engine remembers the previous and current mouse positions, iterates through active objects and calls the collision-checking method for each eligible fruit.

↓

Step 3 — Handle the result

If a fruit is hit, the engine marks it as sliced or removes it, increases the score and updates the game's state. If a bomb is hit, it applies the relevant penalty instead.

A simplified example of the interaction looks like this:

‹/› Python

```python
# Simplified illustration of GameEngine logic

for fruit in fruits:
```

```
    if fruit.intersects_segment(previous_mouse, current_mouse):
        fruit.sliced = True
        score += 1
```

This is illustrative pseudocode; your actual code may use different method names, parameters, score values or object-removal logic.

The important design principle is that the fruit provides the collision test, while the engine controls scoring and gameplay. The engine must also avoid awarding points repeatedly for the same fruit and should handle objects that are already sliced.

4. A simple example you can explain during your lab evaluation

Suppose an apple is at position `(200, 150)` with a collision radius of 20 pixels.

The mouse moves from `(150, 150)` to `(250, 150)` in one frame.

Check	Result
Is the previous mouse position inside the apple?	No
Is the current mouse position inside the apple?	No
Does the line segment pass through the apple?	Yes
What should the game do?	Detect the slice and award a point

The closest point on the segment is `(200, 150)`, which is exactly the apple's center. Therefore, the distance is zero.

Since $0 \leq 20$, the collision is detected.

Try explaining it yourself

Evaluation question: Why is line-segment collision detection better than checking only the current mouse position?

Try answering in your own words before revealing the suggested answer.

Show suggested answer

5. A ready-to-speak explanation

Writing

The original problem in our Fruit Ninja game was that fast mouse swipes could miss fruits. This happens because the game checks mouse positions at discrete intervals. If the mouse moves across a fruit between two frames, both recorded positions might be outside the fruit, so point-based collision detection fails.

To solve this, we use line-segment collision detection. We connect the previous and current mouse positions with a line segment and check whether that segment intersects the fruit's collision boundary.

The Fruit class is responsible for the fruit's properties and collision-checking logic, using a method such as `intersects_segment()`. The GameEngine tracks mouse movement, checks the active fruits against the swipe segment and handles the result. When a fruit is hit, the engine updates the score and removes or marks the fruit as sliced.

For example, if the mouse moves from x-coordinate 150 to x-coordinate 250 and an apple is at x-coordinate 200, both mouse positions can be outside the apple, but the line segment passes through it. Our improved collision check detects the slice.

This makes slicing more reliable, especially during fast mouse movements.

One final point to remember: line-segment collision detection improves the reliability of slicing, but it does not automatically solve every collision problem. The collision boundary, mouse sampling, fruit movement and handling of already-sliced objects can still affect the result.

I want my Fruit Ninja game to display a proper Game Over screen instead of only printing the final score. It should show the final score and wait for the player to choose what to do next. Explain how I can implement this in Pygame without breaking the existing game loop.

Worked for 22s

Implementing a proper Game Over screen in your Fruit Ninja game

For your Fruit Ninja project, the best approach is to **add a Game Over state to the existing game engine rather than stopping the game loop when the player loses**.

Your project structure is:

- `main.py` — runs the Pygame loop.
- `game/game_engine.py` — manages gameplay, score, lives and rendering.
- `game/fruit.py` — defines the fruit objects and collision behavior.

You want the game to display a proper screen showing **GAME OVER**, the final score, and options for what the player can do next.

The important part is that the game should stop updating gameplay while the Game Over screen is displayed, but Pygame must continue processing events. Otherwise, the player won't be able to click a button or close the window.

1. Understand the change

Your existing flow probably resembles this:

Playing the game
Move fruits, detect slices, update score and lives

↓

Lives reach zero

↓

Print final score
The player doesn't get a proper in-game screen.

Change it to this:

Playing the game

↓

Lives reach zero
Set a game-over flag

↓

Game Over screen

GAME OVER

Final Score: 25

Play Again · Quit

Keep the main loop alive so clicks and window events continue to work.

2. Step one: Add a game-over state in `game_engine.py`

A game state tells the program which stage the game is in. For this feature, you only need two states initially:

- `game_over = False` — the player is playing.
- `game_over = True` — the player has lost and the final score is displayed.

In your `GameEngine` class, initialize the flag alongside your existing score and lives variables:

```python
self.game_over = False
```

Then, wherever your existing code detects that the player has run out of lives, set:

```python
if self.lives <= 0:
    self.game_over = True
```

Use your existing lives variable and loss condition; don't create a second lives counter.

Why use a flag instead of exiting the loop?

If you use `break`, `sys.exit()` or stop the main loop as soon as the player loses, the game window can close before the player sees the final score. A flag lets the game transition to another screen while the application remains active.

3. Step two: Draw the Game Over screen

In `game/game_engine.py`, add a method to draw the overlay. The following is an example for a 700 × 600 window.

```python
def draw_game_over(self, screen):
    # Dark overlay
    overlay = pygame.Surface((700, 600), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 190))
    screen.blit(overlay, (0, 0))

    title_font = pygame.font.Font(None, 64)
    score_font = pygame.font.Font(None, 40)
```

```
    option_font = pygame.font.Font(None, 30)

    title = title_font.render(
        "GAME OVER", True, (255, 80, 80)
    )
    score = score_font.render(
        f"Final Score: {self.score}", True, (255, 255, 255)
    )
    replay = option_font.render(
        "Press R to Play Again", True, (100, 255, 150)
    )
    quit_text = option_font.render(
```

This draws a semi-transparent dark layer over the existing game, followed by the title, final score and instructions.

The overlay makes the Game Over screen readable without needing to create a separate Pygame window.

This example assumes that your engine has `self.score` and that you pass the display surface to the method. Adapt the method signature if your engine already stores its display surface.

4. Step three: Keep the main game loop running

This is the most important change.

Your loop in `main.py` should continue handling events regardless of whether the player is playing or viewing the Game Over screen.

A simplified example:

Python

```python
running = True

while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

        if game.game_over:
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    running = False

                elif event.key == pygame.K_r:
                    game.reset()
```

```
    if not game.game_over:
        game.update()

    game.render()
```

Here, `game` represents your existing `GameEngine` instance, and `screen` represents the Pygame display surface. Adjust the method calls to match your actual project.

The key distinction is:

Python

```python
if not game.game_over:
    game.update()
```

When the player loses, gameplay updates stop, so fruits don't keep moving and the score doesn't keep changing. However, the event loop still runs, allowing the player to choose what to do next.

If your existing `render()` method calls `pygame.display.flip()` internally, don't add a second display update unnecessarily; preserve your current rendering structure.

5. Step four: Implement Play Again properly

If you display a Play Again option, it must reset the game rather than just clear `game_over`. Otherwise, the player might restart with zero lives, old fruits, or stale collision information.

Add a reset method to `GameEngine`:

Python

```python
def reset(self):
    self.score = 0
    self.lives = 3
    self.game_over = False

    self.fruits.clear()
    self.previous_mouse_pos = None

    # Reset other existing gameplay variables here:
    # spawn timers, difficulty, blade trail, etc.
```

This is an example, not a drop-in replacement. Use the actual names of your fruit list, lives counter and mouse-position variables. Reset any other state that your existing game uses.

For instance, if your engine tracks elapsed time or fruit spawn intervals, those should also return to their initial values. If your game has a different starting number of lives, preserve that value.

Important: Don't replace your entire `GameEngine` class just to add this feature. Add the flag, drawing method and reset behavior to the existing class so your fruit movement and collision detection remain intact.

6. Test your implementation

Use this checklist after making the changes.

Testing checklist 0 of 7

- [ ] Start the game and confirm normal gameplay still works.
- [ ] Use up all lives and confirm the Game Over screen appears.
- [ ] Check that the final score matches the score earned during gameplay.
- [ ] Wait on the Game Over screen and confirm fruits and score no longer update.
- [ ] Press R and verify that a fresh game starts with the correct score and lives.
- [ ] Press ESC and verify that the game closes cleanly.
- [ ] Check that closing the window still works from both gameplay and Game Over.

What to explain during your lab evaluation

You can describe your implementation like this:

Writing

Previously, when the player ran out of lives, the game only printed the final score. I improved this by introducing a Game Over state in the existing game engine.

When the number of lives reaches zero, the game sets the game-over flag. The main game loop continues running, but it stops updating gameplay. Instead, the rendering logic displays a Game Over screen with the final score and options to play again or quit.

I also added a reset mechanism so that selecting Play Again resets the score, lives, active fruits

and other relevant gameplay state. This approach preserves the existing game loop and collision detection while making the game more user-friendly.

My recommendation is to implement the **Game Over flag and screen first**, test that it works, and then add Play Again. That makes it easier to identify problems without disturbing the rest of your working game.

> How can I add Easy, Medium, and Hard difficulty levels to my Fruit Ninja game? I want each level to have a different fruit spawn rate and bomb probability. Explain how to structure the difficulty settings and allow the player to restart the game with a selected difficulty.

Adding Easy, Medium, and Hard modes to your Fruit Ninja game

The cleanest way to implement difficulty levels is to keep your existing `main.py`, `game_engine.py`, and `fruit.py` structure, and let `GameEngine` **manage the selected difficulty**.

Each difficulty should control two things:

- **Fruit spawn rate:** how frequently new fruits appear.
- **Bomb probability:** how likely a spawned object is to be a bomb instead of a fruit.

You should also integrate difficulty selection with your Game Over screen, so the player can restart using the same difficulty or choose another one.

1. Decide the difficulty settings

Start with a central configuration rather than scattering different numbers throughout your code.

These are example values you can tune after testing:

Difficulty	Spawn interval	Bomb probability	Gameplay
Easy	1.2 seconds	5%	Fewer objects and fewer bombs
Medium	0.8 seconds	12%	Balanced challenge
Hard	0.5 seconds	22%	Frequent spawns and more bombs

The spawn interval is the time between spawn attempts. A smaller interval means more frequent attempts. Bomb probability is the chance that an individual spawn attempt creates a bomb, assuming each attempt creates either a bomb or a fruit.

These are starting values, not measurements from your existing game. If your project already has a working spawn system, use its existing timing mechanism and adapt the values.

2. Structure the difficulty settings

In `game/game_engine.py`, define a configuration dictionary near the top of the file, outside the `GameEngine` class.

Python

```python
DIFFICULTY_SETTINGS = {
    "Easy": {
        "spawn_interval": 1.2,
        "bomb_probability": 0.05,
    },
    "Medium": {
        "spawn_interval": 0.8,
        "bomb_probability": 0.12,
    },
    "Hard": {
        "spawn_interval": 0.5,
        "bomb_probability": 0.22,
    },
}
```

This is better than repeatedly writing conditions such as `if difficulty == "Easy"` throughout the code. You can change the settings in one place, and the engine can retrieve the appropriate values for the selected mode.

3. Store the selected difficulty in `GameEngine`

When a game starts, the engine should remember the selected difficulty and load its settings.

Add or adapt this logic in your `GameEngine` class:

Python

```python
class GameEngine:
    def __init__(self, difficulty="Medium"):
        self.difficulty = difficulty
        self.apply_difficulty()

        self.score = 0
        self.lives = 3
        self.game_over = False
```

```
        # Keep your existing initialization here:
        # fruit list, spawn timer, mouse positions, etc.

    def apply_difficulty(self):
        settings = DIFFICULTY_SETTINGS[self.difficulty]

        self.spawn_interval = settings["spawn_interval"]
        self.bomb_probability = settings["bomb_probability"]
```

If your existing constructor already initializes the score, lives, fruit list, and other variables, **keep that code**. Add the difficulty handling to it instead of replacing the constructor.

The `apply_difficulty()` method loads the values for the chosen mode. For example, selecting Hard sets the spawn interval to `0.5` seconds and the bomb probability to `0.22`.

4. Use the settings when spawning objects

Next, find the existing code in `game_engine.py` that decides when to spawn fruits and bombs.

A. Control the spawn interval

If your game already uses elapsed time or a timer, use `self.spawn_interval` in that existing logic.

For a timer-based implementation, the logic could look like this:

Python

```
self.spawn_timer += dt

if self.spawn_timer >= self.spawn_interval:
    self.spawn_object()
    self.spawn_timer = 0
```

Here, `dt` is the elapsed time in seconds since the previous frame.

Make sure `dt` is actually passed into your update method if your game uses this approach. If the existing project uses a frame counter or Pygame timer, adapt that mechanism instead of introducing a second, competing spawn timer.

B. Control the bomb probability

Use Python's `random.random()` to choose whether a spawn attempt creates a bomb.

Python

```python
import random

def spawn_object(self):
    if random.random() < self.bomb_probability:
        # Create a bomb using your existing bomb logic
        self.spawn_bomb()
    else:
        # Create a fruit using your existing fruit logic
        self.spawn_fruit()
```

The `spawn_bomb()` and `spawn_fruit()` calls above represent your existing object-creation logic; use your actual method names or adapt the current spawning code.

For example, if `bomb_probability` is `0.22`, a random value below `0.22` produces a bomb. Otherwise, the spawn attempt produces a fruit.

Important: If your current spawning system creates multiple fruits in a single wave, decide whether bomb probability applies per object or per wave. The example above applies it per spawn attempt.

5. Allow the player to select a difficulty

A simple approach for your lab project is to add a difficulty selection screen before gameplay begins.

You can support these keyboard controls:

- `1` — Easy
- `2` — Medium
- `3` — Hard

For a first implementation, keyboard selection is easier to integrate safely than adding clickable buttons. You can add mouse-click buttons later.

In `main.py`, handle the selection through Pygame's existing event loop:

Python

```python
if event.type == pygame.KEYDOWN:
    if event.key == pygame.K_1:
        game.start_new_game("Easy")

    elif event.key == pygame.K_2:
        game.start_new_game("Medium")
```

```
    elif event.key == pygame.K_3:
        game.start_new_game("Hard")
```

This is illustrative: place these handlers in the appropriate menu-state branch of your existing event loop, rather than handling them unconditionally during gameplay.

You will need a menu screen that displays the three choices. For example:



Conceptual mockup of the selection screen

For clean state management, your game can use three states:

- `MENU` — waiting for the player to choose a difficulty.
- `PLAYING` — running the actual game.
- `GAME_OVER` — displaying the final score and restart options.

This is more flexible than using only a `game_over` Boolean, because you can distinguish the starting menu from the active game and the final screen.

6. Restart the game with the selected difficulty

When the player chooses a difficulty, the engine should reset the game and load the corresponding settings.

Add a method like this to `GameEngine`:

‹/› **Python**

```python
def start_new_game(self, difficulty):
    self.difficulty = difficulty
    self.apply_difficulty()

    self.score = 0
    self.lives = 3
    self.game_over = False

    # Reset your existing gameplay state:
    # - Clear active fruits and bombs
    # - Reset spawn timer
    # - Reset mouse history and blade trail
    # - Reset other round-specific variables
```

Again, retain the existing reset logic in your project and add the difficulty updates to it. Use the actual names of your variables.

When restarting from Game Over, you can provide two options:

- **Play Again:** restart with `self.difficulty`, keeping the current mode.
- **Change Difficulty:** return to the menu and let the player choose another mode.

For example:

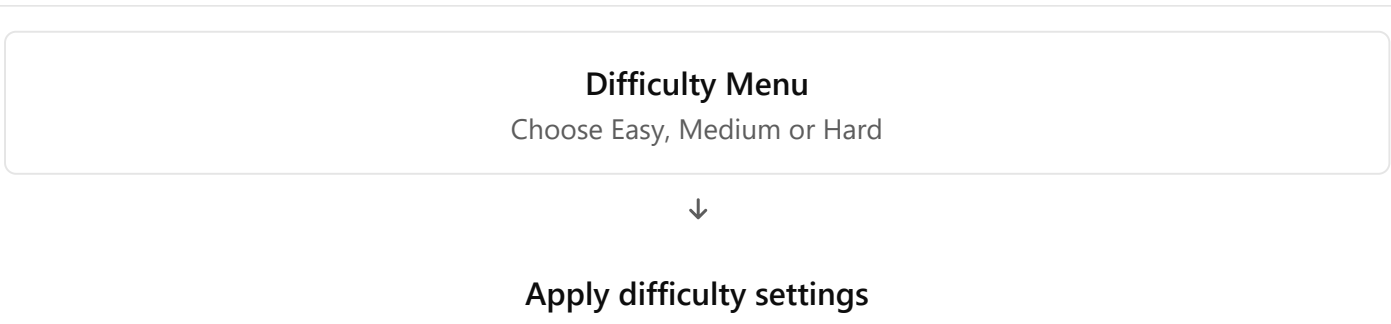
</> Python

```python
# Play Again
game.start_new_game(game.difficulty)

# Change Difficulty
game.state = "MENU"
```

If you use a state-based design, update your main loop so it only calls gameplay updates in the `PLAYING` state. The menu and Game Over states should continue handling Pygame events and rendering their respective screens.

7. How the complete flow works



Load spawn interval and bomb probability

↓

Gameplay

Spawn objects, detect slices, update score and lives

↓

Game Over

Display final score and restart options

↑ Play Again keeps the difficulty; Change Difficulty returns to the menu.

8. Final checklist

Before submitting your project, verify that:

- ○ Easy, Medium and Hard all start correctly.
- ○ Each mode uses its own spawn interval and bomb probability.
- ○ The difficulty setting doesn't change unexpectedly during gameplay.
- ○ Play Again resets score, lives, objects and timers.
- ○ Play Again preserves the selected difficulty.
- ○ Change Difficulty returns to the selection menu.
- ○ Your existing slicing and collision detection still work.
- ○ The window can still be closed normally.

My recommendation: implement the settings dictionary first, connect it to your existing spawn logic, and then add the selection menu and restart behavior. This keeps the change modular and reduces the risk of breaking your existing Fruit Ninja game loop.

How can I add separate sound effects for slicing fruit, hitting a bomb, and Game Over in Pygame? Explain how pygame.mixer.Sound works and how I can create or load sounds and trigger them at the correct events. Also explain how to avoid the sounds overlapping too much.

Adding sound effects to your Fruit Ninja game

For your Fruit Ninja project, you can add three separate sound effects:

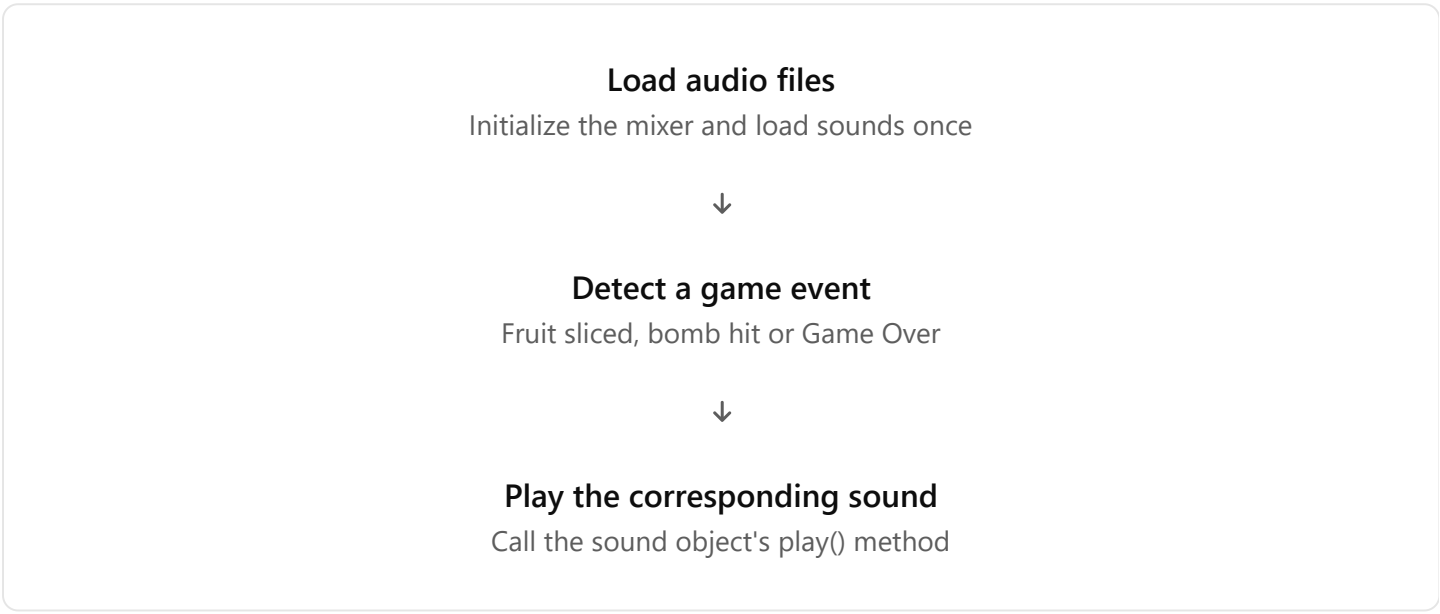
- **Fruit sliced:** a short slicing or popping sound.
- **Bomb hit:** a low explosion sound.
- **Game Over:** a short defeat sound or descending tone.

The best approach is to load the sounds once when the game starts, then play each sound only when its corresponding event occurs. You should integrate this with your existing `main.py` and `game_engine.py` without changing your fruit collision detection.

1. How `pygame.mixer.Sound` works

Pygame's mixer module handles audio playback. The `pygame.mixer.Sound` class loads a short sound effect into memory so that it can be played when needed.

The basic workflow is:



For example:

Python

```python
pygame.mixer.init()

slice_sound = pygame.mixer.Sound("assets/slice.wav")
slice_sound.play()
```

Here:

- `pygame.mixer.init()` initializes audio playback.
- `pygame.mixer.Sound(...)` loads a sound file.
- `.play()` starts playback.

Important: don't load a new `Sound` object every time a fruit is sliced. Load the sounds once and reuse them.

2. Organize your sound files

Inside your Fruit Ninja repository, create an `assets` folder for the audio files.

Plain text

```
LAB4_FruitNinja/
├── main.py
├── game/
│   ├── game_engine.py
│   └── fruit.py
└── assets/
    ├── slice.wav
    ├── bomb.wav
    └── game_over.wav
```

You can use `.wav` or supported `.ogg` files. WAV is a straightforward choice for short sound effects.

You have two options for obtaining sounds:

- **Create your own:** record a short slicing sound, make a pop using a basic audio editor, or synthesize a tone.
- **Use existing sound effects:** download appropriately licensed audio from a source such as Freesound ↗, and check the license and attribution requirements before including it in your submission.

Keep the audio files inside your project so the game doesn't depend on an external file path on your computer.

3. Load all sounds when the game starts

A good place for audio initialization is in `main.py` or in a dedicated sound manager that your game engine receives.

For a small project, a simple dictionary is sufficient.

Python

```python
import pygame
from pathlib import Path

pygame.mixer.init()

ASSET_DIR = Path(__file__).resolve().parent / "assets"
```

```
sounds = {
    "slice": pygame.mixer.Sound(ASSET_DIR / "slice.wav"),
    "bomb": pygame.mixer.Sound(ASSET_DIR / "bomb.wav"),
    "game_over": pygame.mixer.Sound(ASSET_DIR / "game_over.wav"),
}
```

This assumes `assets` is beside `main.py`. If your project has a different structure, adjust the path accordingly.

You can also set each sound's volume:

Python

```
sounds["slice"].set_volume(0.45)
sounds["bomb"].set_volume(0.65)
sounds["game_over"].set_volume(0.7)
```

Volume ranges from `0.0` (silent) to `1.0` (maximum).

Load the sounds after initializing Pygame's mixer and before starting gameplay. If you already initialize the mixer elsewhere, don't initialize it a second time unnecessarily.

4. Trigger the slicing sound at the correct event

In your game, the sound should play when the collision detection confirms that a fruit has been sliced, not every time the fruit is drawn or updated.

Your `game_engine.py` already handles mouse swipes and fruit collisions. Add the sound call to that existing collision-handling block.

Python

```
if fruit.intersects_segment(previous_mouse, current_mouse):
    if not fruit.sliced:
        fruit.sliced = True
        self.score += 1
        self.sounds["slice"].play()
```

This illustrates the intended logic. Adapt the condition and variable names to your actual code.

The `not fruit.sliced` check is important: if the same fruit is detected again on the next frame, it should not award another point or replay the slicing sound.

To make this work, pass the sound dictionary to the engine when you create it, or store the sound objects in a sound manager accessible to the engine.

For example, in `main.py`:

Python

```python
game = GameEngine(sounds=sounds)
```

Then in the engine constructor:

Python

```python
def __init__(self, sounds):
    self.sounds = sounds
    # Keep the rest of your existing initialization
```

If your constructor already accepts other arguments, preserve them.

5. Trigger the bomb sound

The bomb sound should play only when the player actually hits a bomb, not whenever a bomb appears on screen.

In the bomb collision-handling branch:

Python

```python
if bomb.intersects_segment(previous_mouse, current_mouse):
    if not bomb.hit:
        bomb.hit = True
        self.sounds["bomb"].play()

        # Apply your existing bomb penalty
```

The `bomb.hit` flag is illustrative; use your project's existing mechanism for marking or removing a hit bomb.

The distinction matters because **spawning a bomb and hitting a bomb are different events**. The sound belongs in the collision response.

6. Trigger Game Over only once

The Game Over sound should play when the game transitions from active gameplay to the Game Over state.

For example:

Python

```python
if self.lives <= 0 and not self.game_over:
    self.game_over = True
    self.sounds["game_over"].play()
```

The `not self.game_over` condition prevents the sound from restarting on every frame after the player loses.

If you already have a method that handles the transition to Game Over, put the sound call inside that method instead. This keeps the transition logic in one place.

7. How to prevent sound effects from overlapping too much

This is particularly important for Fruit Ninja because a player can slice several fruits in quick succession.

By default, calling `.play()` repeatedly on a `Sound` can create overlapping playback. Pygame also uses mixer channels to manage simultaneous sounds.

There are three useful techniques.

Method A: Use dedicated mixer channels

Assign each type of sound its own channel. Playing a new sound on the same channel replaces the sound already playing on that channel.

Python

```python
pygame.mixer.set_num_channels(8)

slice_channel = pygame.mixer.Channel(0)
bomb_channel = pygame.mixer.Channel(1)
game_over_channel = pygame.mixer.Channel(2)
```

Then play sounds through those channels:

```python
slice_channel.play(sounds["slice"])
bomb_channel.play(sounds["bomb"])
game_over_channel.play(sounds["game_over"])
```

For example, if three fruits are sliced very quickly, the slicing sound can restart on channel 0 rather than creating several overlapping copies. The bomb and Game Over sounds use separate channels.

Method B: Add a cooldown for slicing sounds

A cooldown prevents the slicing sound from being triggered too frequently.

For example, you might allow one slicing sound every 0.08 seconds:

```python
# Initialize in your engine
self.last_slice_sound_time = 0
self.slice_sound_cooldown = 80  # milliseconds
```

Then, inside the successful fruit collision block:

```python
now = pygame.time.get_ticks()

if now - self.last_slice_sound_time >= self.slice_sound_cooldown:
    slice_channel.play(self.sounds["slice"])
    self.last_slice_sound_time = now
```

This limits how often the sound is restarted. It doesn't prevent every possible overlap if other channels are involved, but it helps avoid an excessively noisy sequence of slicing sounds.

Method C: Control volume

Different sound effects have different perceived loudness. Keep the slicing sound relatively soft and reserve more emphasis for bomb hits and Game Over.

For example:

Sound	Example volume
Slice	0.40
Bomb	0.65
Game Over	0.70

Adjust these values by testing the game on your computer. These are starting points, not fixed requirements.

My recommendation: use dedicated channels, a small slicing cooldown, and sensible volume levels. That gives you predictable playback without needing a complicated audio system.

8. Final implementation checklist

Audio integration 0/8

- [ ] Create the three audio files in the assets folder.
- [ ] Initialize the mixer and load each sound once.
- [ ] Play the slice sound only when a fruit is successfully sliced.
- [ ] Play the bomb sound only when a bomb is hit.
- [ ] Play the Game Over sound only on the transition to Game Over.
- [ ] Use dedicated channels or a cooldown to limit excessive overlap.
- [ ] Test that audio paths work when running main.py from the project folder.
- [ ] Confirm the game still works if audio initialization or an asset fails.

If you want the game to remain playable on systems without working audio, you can handle mixer initialization errors and missing sound files gracefully rather than letting them crash the application.

The central principle is simple: **load sounds once, trigger them at the exact gameplay event, and manage playback through channels or cooldowns.** This keeps the audio responsive while preserving your existing game loop and collision detection.

I have to submit my updated Python game through my personal GitHub repository. Explain how to organize the project files, update README.md, check requirements.txt, commit and push the

changes, and make sure I do not accidentally submit a pull request to the original repository.

Final submission guide: Fruit Ninja on your personal GitHub

Your goal is to submit the updated Fruit Ninja game to **your personal repository**, not the original assigned repository.

- Original repository: SETAPESU26/41_fruit-ninja ↗
- Your personal repository: SruthiVejju/LAB4_FruitNinja_SruthiVejju_PES1UG24CS472- ↗

The safest workflow is to check your local files, verify the Git remote, test the game, commit your changes, and push directly to your personal repository.

Step 1: Organize your project files

Your repository should contain the source code, required assets and documentation needed to run the game.

A suggested structure is:

LAB4_FruitNinja_SruthiVejju_PES1UG24CS472-/

- **main.py**
  Main entry point and game loop
- **game/**
  Game engine and fruit classes
- **assets/**
  Fruit images, backgrounds and sound effects
- **requirements.txt**
  Python package dependencies
- **README.md**
  Project description and running instructions
- **.gitignore**
  Files that Git should ignore

Your actual project may have a different structure. **Keep the existing working structure** and include any modules, configuration files or assets that the game actually uses. Do not rename files just to match this example.

Make sure your updated features are integrated before submission:

- Game Over screen with final score and restart/quit options.

- Easy, Medium and Hard difficulty settings, if implemented.
- Separate slicing, bomb and Game Over sounds, if implemented.
- Existing fruit movement, slicing and collision detection.

Do not include virtual environments such as `venv/`, Python cache directories, or unnecessary generated files.

Step 2: Check `requirements.txt`

Open `requirements.txt` in VS Code. It should list the external Python packages needed to run your game.

If Pygame is the only external package your game uses, the file can simply contain:

Plain text

```
pygame
```

If your project uses other third-party packages, list those too, with appropriate version constraints if needed.

Do not add Python's built-in modules, such as `random`, `pathlib`, or `sys`, because they do not need to be installed separately.

To test the dependencies, open a terminal in your project directory and run:

Bash

```
python -m pip install -r requirements.txt
python main.py
```

If you have a virtual environment, activate it first. Confirm that the game launches and that its required images and sounds load correctly.

Step 3: Update `README.md`

Your README should tell the evaluator what the game does, how to install its dependencies, how to run it, and how to play it.

Here is a submission-ready starting point. Edit the feature list to include only features actually implemented in your final version.

Writing

Fruit Ninja Game — Python and Pygame

Overview

This project is a Fruit Ninja-style 2D game developed using Python and Pygame. The player slices fruits using mouse movements, earns points and avoids bombs.

Features

- Fruit spawning and movement with gravity.
- Mouse-based slicing and collision detection.
- Score and lives tracking.
- Game Over screen displaying the final score.
- Restart and quit options.
- Easy, Medium and Hard difficulty levels.
- Separate sound effects for fruit slicing, bomb hits and Game Over.

Note: Retain only the features implemented in the submitted version.

Project Structure

- `main.py` — initializes and runs the game.
- `game/` — contains the game engine and fruit-related classes.
- `assets/` — contains the images, backgrounds and sound effects used by the game.
- `requirements.txt` — lists the required Python packages.

Requirements

- Python 3
- Pygame

Installation

1. Clone or download this repository.
2. Open a terminal in the project directory.
3. Install the required packages:

```bash
python -m pip install -r requirements.txt
```

How to Run

Run the following command from the project directory:

```bash
python main.py
```

How to Play

- Move the mouse across fruits to slice them.
- Earn points by slicing fruits.
- Avoid bombs.
- Watch your remaining lives.
- Select a difficulty if the difficulty menu is available.
- When the game ends, view your final score and choose whether to restart or quit.

Technologies Used

- Python
- Pygame

Author

Sruthi Vejju

Repository

Personal repository:
https://github.com/SruthiVejju/LAB4_FruitNinja_SruthiVejju_PES1UG24CS472-

Before saving, check that the directory names, controls, difficulty-selection behavior and restart instructions match your actual code.

Step 4: Check which GitHub repository your local project points to

This is the most important step for avoiding accidental submission to the original repository.

In VS Code, open **Terminal** → **New Terminal**, then run:

Bash

```
git remote -v
```

You should see output similar to:

Plain text

```
origin  https://github.com/SruthiVejju/LAB4_FruitNinja_SruthiVejju_PES1UG24CS472-.git (fetch)
origin  https://github.com/SruthiVejju/LAB4_FruitNinja_SruthiVejju_PES1UG24CS472-.git (push)
```

Check that the push URL points to **your personal repository**.

If `origin` points to the original assigned repository instead, change it:

Bash

```
git remote set-url origin https://github.com/SruthiVejju/LAB4_FruitNinja_SruthiVejju_PES1UG24CS472-.git
```

Then verify again:

Bash

```
git remote -v
```

If the original repository is configured as `upstream`, that is okay. You can leave it there, provided you push to your personal `origin`.

Step 5: Check your changes before committing

Run:

Bash

```
git status
```

This shows modified files, new files and untracked files.

Review the list carefully. Make sure your updated Python files, README, requirements and required assets are included.

Then inspect the changes:

Bash

```
git diff
```

For new files that Git has not started tracking yet, `git diff` won't show their contents until they are staged.

Before proceeding, check that you are not submitting:

- A virtual environment such as `venv/`.
- Cache files such as `__pycache__/` or `.pyc` files.
- Unnecessary IDE configuration or temporary files.
- Personal credentials, tokens or secrets.
- Large, unnecessary datasets or generated files.

A basic `.gitignore` for Python could be:

gitignore

```
__pycache__/
*.py[cod]
venv/
.venv/
.env
```

Keep any required game assets tracked. Do not ignore your `assets/` folder if the game needs it to run.

Step 6: Commit your updated game

Once you have tested the game and reviewed your changes, stage the files:

Bash

```
git add .
```

Check what is staged:

Bash

```
git status
git diff --cached
```

If everything looks correct, create a commit:

Bash

```
git commit -m "Add game improvements and update documentation"
```

A commit is a saved snapshot of your changes in Git. It does not upload the changes to GitHub yet.

Step 7: Push to your personal repository

First, check your current branch:

Bash

```
git branch --show-current
```

If you are working on `main`, and your personal repository uses `main`, run:

Bash

```
git push -u origin main
```

If your current branch has a different name, push that branch instead, for example:

Bash

```
git push -u origin feature-updates
```

Use the actual branch name reported by `git branch --show-current`.

If Git reports that the remote contains changes you don't have locally, do not immediately force-push. Inspect the situation and integrate the remote changes safely.

Step 8: Verify the submission on GitHub

Open your personal repository:

Open your personal Fruit Ninja repository ↗

Confirm that:

- [ ] The latest commit is visible.
- [ ] `main.py` and all required Python modules are present.
- [ ] `README.md` displays correctly.
- [ ] `requirements.txt` is present.
- [ ] Required images and sound files are uploaded.
- [ ] The updated Game Over, difficulty and audio features work when tested.
- [ ] No unnecessary files or secrets were committed.

How to avoid accidentally creating a pull request

A normal `git push` to your personal repository does **not** automatically create a pull request to the original repository. A pull request is a separate action.

To keep your submission on your personal repository:

1. Verify that `origin` points to your personal repository before pushing.
2. Push your branch to `origin`, not to `upstream` or the original repository.
3. Do not click **Compare & pull request** or submit a pull request to `SETAPESU26/41_fruit-ninja` unless your instructor explicitly asks you to do so.

Your final submission link should be your personal repository URL, not a pull request URL.

Recommended order: test the game → update README and requirements → verify `git remote -v` → commit → push → open your personal repository and verify the uploaded files.